UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Heaps
Semana 7

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

21 de septiembre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Árbol binario **completo**: qué es, y por qué importa aquí

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Árbol binario **completo**: qué es, y por qué importa aquí
- La propiedad de *Heap* (min-heap / max-heap)

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Árbol binario **completo**: qué es, y por qué importa aquí
- La propiedad de *Heap* (min-heap / max-heap)
- Representar un heap con un simple **arreglo** — sin punteros

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Árbol binario **completo**: qué es, y por qué importa aquí
- La propiedad de *Heap* (min-heap / max-heap)
- Representar un heap con un simple **arreglo** — sin punteros
- El TDA Heap / Cola de Prioridad, y la complejidad de sus operaciones

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada: BST

Un BST está diseñado para **buscar** rápido: la propiedad de orden total (izquierda $<$ raíz $<$ derecha) permite descartar la mitad del árbol en cada paso.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada: BST

Un BST está diseñado para **buscar** rápido: la propiedad de orden total (izquierda $<$ raíz $<$ derecha) permite descartar la mitad del árbol en cada paso.

¿Y si lo único que necesitas es encontrar **rápido** el mínimo (o máximo), sin necesitar buscar un valor arbitrario?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La clase pasada: BST

Un BST está diseñado para **buscar** rápido: la propiedad de orden total (izquierda $<$ raíz $<$ derecha) permite descartar la mitad del árbol en cada paso.

¿Y si lo único que necesitas es encontrar **rápido** el mínimo (o máximo), sin necesitar buscar un valor arbitrario?

Menos exigencia, más eficiencia

Si renunciamos al orden total y solo exigimos una condición **local** y más débil, podemos mantener esa estructura **perfectamente balanceada** — gratis, sin rotaciones — y hasta representarla sin punteros.

Árbol binario completo

Recordando: tipos de árbol binario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuál de los tipos de árbol binario garantiza altura $O(\lg n)$ **siempre**, sin importar el orden de inserción?

La clase pasada vimos árboles llenos, completos, perfectos y degenerados.

Recordando: tipos de árbol binario

¿Cuál de los tipos de árbol binario garantiza altura $O(\lg n)$ **siempre**, sin importar el orden de inserción?

La clase pasada vimos árboles llenos, completos, perfectos y degenerados.

Árbol binario completo

Todos los niveles están completamente llenos, **excepto posiblemente el último**, que se llena **de izquierda a derecha**. Con n nodos, la altura es exactamente $\lfloor \lg n \rfloor$ — no hay forma de que degenere.

Por qué esto es la clave

Un heap va a ser, **siempre**, un árbol binario completo — eso es lo que le da su altura $O(\lg n)$ garantizada, sin necesitar ningún mecanismo de balanceo adicional (a diferencia de lo que vendrá con árboles AVL más adelante en el curso).

La propiedad de Heap

Min-Heap y Max-Heap

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Propiedad de Min-Heap

Para todo nodo v distinto de la raíz: $valor(padre(v)) \leq valor(v)$. Es decir, cada padre es **menor o igual** que sus hijos.

Min-Heap y Max-Heap

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Propiedad de Min-Heap

Para todo nodo v distinto de la raíz: $valor(padre(v)) \leq valor(v)$. Es decir, cada padre es **menor o igual** que sus hijos.

(Max-Heap: la desigualdad se invierte — cada padre es **mayor o igual** que sus hijos. Todo lo de hoy aplica igual a ambos, solo cambiando la dirección de la comparación.)

¿Qué garantiza, y qué NO garantiza?

En un min-heap, ¿dónde está **garantizado** que esté el elemento mínimo? ¿Y el segundo menor — también en una posición fija?

¿Qué garantiza, y qué NO garantiza?

En un min-heap, ¿dónde está **garantizado** que esté el elemento mínimo? ¿Y el segundo menor — también en una posición fija?

Respuesta

El mínimo **siempre** está en la raíz — eso sí está garantizado. Pero el segundo menor puede estar en **cualquiera** de los dos hijos de la raíz (o más abajo en ese subárbol): la propiedad de heap es solo **local** (padre vs. hijos), no un orden total como en BST.

¿Qué garantiza, y qué NO garantiza?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

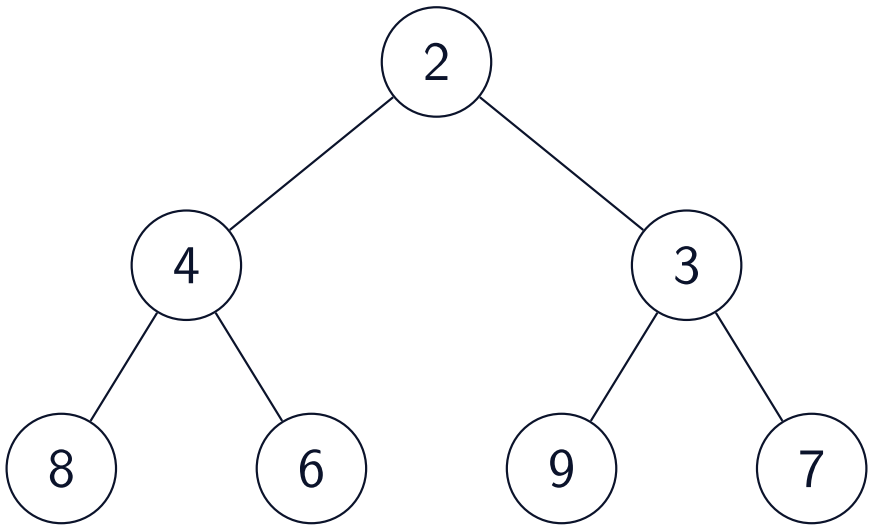
En un min-heap, ¿dónde está **garantizado** que esté el elemento mínimo? ¿Y el segundo menor — también en una posición fija?

Respuesta

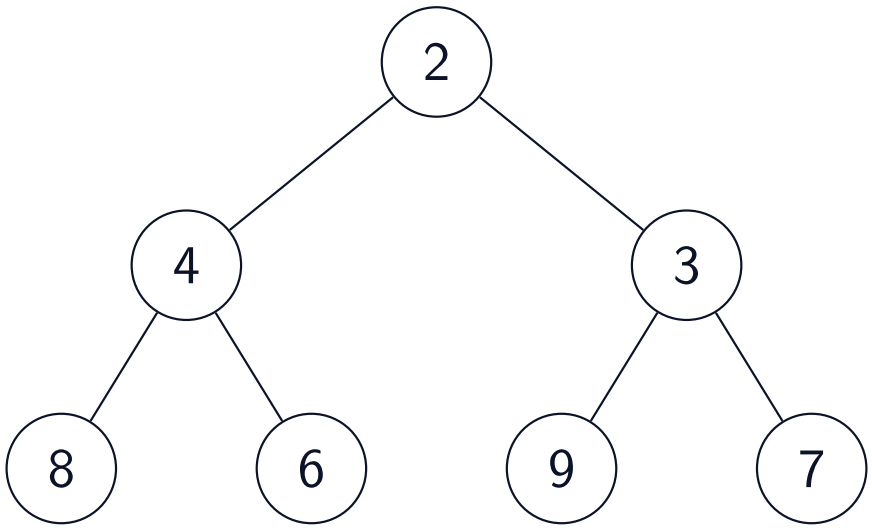
El mínimo **siempre** está en la raíz — eso sí está garantizado. Pero el segundo menor puede estar en **cualquiera** de los dos hijos de la raíz (o más abajo en ese subárbol): la propiedad de heap es solo **local** (padre vs. hijos), no un orden total como en BST.

Ese es precisamente el "precio" que se paga por ganar altura garantizada: un heap **no** sirve para buscar un valor arbitrario en $O(\lg n)$ — solo para acceder al extremo (mín. o máx.).

Un ejemplo



Un ejemplo



Min-heap válido: cada padre $\leq$ sus hijos. Nota que 4 (izquierda de la raíz) es mayor que 3 (derecha de la raíz) — **no** hay orden entre hermanos ni entre subárboles, solo entre cada nodo y su padre directo.

Representación con arreglo

¿Punteros, o algo más simple?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Eso te recuerda a alguna estructura que ya conoces bien?

Un árbol binario completo siempre se llena "en el mismo orden" (nivel por nivel, izquierda a derecha).

¿Punteros, o algo más simple?

¿Eso te recuerda a alguna estructura que ya conoces bien?

Un árbol binario completo siempre se llena "en el mismo orden" (nivel por nivel, izquierda a derecha).

Un arreglo, recorrido por niveles

Si numeramos los nodos $1, 2, 3, \ldots$ en ese orden (BFS por niveles) y los guardamos en un arreglo H en esas posiciones, **no hace falta ningún puntero**: la posición misma codifica la estructura del árbol.

Las fórmulas de índices: dos convenciones válidas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Relación	**0-indexado**	**1-indexado**
Padre de $H[i]$	$\lfloor (i-1)/2 \rfloor$	$\lfloor i/2 \rfloor$
Hijo izquierdo de $H[i]$	$2i+1$	$2i$
Hijo derecho de $H[i]$	$2i+2$	$2i+1$

Las fórmulas de índices: dos convenciones válidas

Relación	**0-indexado**	**1-indexado**
Padre de $H[i]$	$\lfloor (i-1)/2 \rfloor$	$\lfloor i/2 \rfloor$
Hijo izquierdo de $H[i]$	$2i+1$	$2i$
Hijo derecho de $H[i]$	$2i+2$	$2i+1$

Son la **misma** idea, solo cambia si la raíz vive en $H[0]$ o en $H[1]$ — ambas convenciones son igual de válidas, y en ambas todas las operaciones son aritmética de enteros $O(1)$.

Las fórmulas de índices: dos convenciones válidas

Relación	**0-indexado**	**1-indexado**
Padre de $H[i]$	$\lfloor (i-1)/2 \rfloor$	$\lfloor i/2 \rfloor$
Hijo izquierdo de $H[i]$	$2i+1$	$2i$
Hijo derecho de $H[i]$	$2i+2$	$2i+1$

Son la **misma** idea, solo cambia si la raíz vive en $H[0]$ o en $H[1]$ — ambas convenciones son igual de válidas, y en ambas todas las operaciones son aritmética de enteros $O(1)$.

De aquí en adelante (y en la implementación de la próxima clase) usaremos **1-indexado**.

El mismo ejemplo, como arreglo

Índice	1	2	3	4	5	6
$H[\,]$	2	4	3	8	6	9

(falta $H[7] = 7$)

El mismo ejemplo, como arreglo

Índice	1	2	3	4	5	6
$H[\,]$	2	4	3	8	6	9

(falta $H[7] = 7$)

Verifica: padre de $H[4] = 8$ es $H[\lfloor 4/2 \rfloor] = H[2] = 4$ — coincide con el árbol de la diapositiva anterior. La estructura completa vive **solo** en las posiciones del arreglo.

El TDA Heap

Operaciones del TDA



Operaciones del TDA

- VER-EXTREMO: consultar el mínimo (o máximo) sin eliminarlo.

Operaciones del TDA

- VER-EXTREMO: consultar el mínimo (o máximo) sin eliminarlo.
- INSERTAR(x): agregar un nuevo elemento, preservando la propiedad de heap.

Operaciones del TDA

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- VER-EXTREMO: consultar el mínimo (o máximo) sin eliminarlo.
- INSERTAR(x): agregar un nuevo elemento, preservando la propiedad de heap.
- EXTRAER-EXTREMO: eliminar y devolver el mínimo (o máximo), preservando la propiedad de heap.

Operaciones del TDA

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- VER-EXTREMO: consultar el mínimo (o máximo) sin eliminarlo.
- INSERTAR(x): agregar un nuevo elemento, preservando la propiedad de heap.
- EXTRAER-EXTREMO: eliminar y devolver el mínimo (o máximo), preservando la propiedad de heap.
- CONSTRUIR-HEAP: convertir un arreglo arbitrario de n elementos en un heap válido.

Complejidad — panorama general

Operación	Complejidad
Ver-Extremo	$O(1)$
Insertar	$O(\lg n)$
Extraer-Extremo	$O(\lg n)$
Construir-Heap (desde n elementos)	$O(n)$

Complejidad — panorama general

Operación	Complejidad
Ver-Extremo	$O(1)$
Insertar	$O(\lg n)$
Extraer-Extremo	$O(\lg n)$
Construir-Heap (desde n elementos)	$O(n)$

Las tres primeras se derivan directamente de la altura $O(\lg n)$ garantizada. Construir-Heap en $O(n)$ (no $O(n \lg n)$) es una sorpresa que vamos a demostrar en la próxima clase, cuando veamos la implementación completa.

Comparando con lo que ya conoces

	Arreglo	**BST**	**Heap**
Ver mínimo	$O(n)$	$O(h)$	$O(1)$
Insertar	$O(1)/O(n)$	$O(h)$	$O(\lg n)$
Extraer mínimo	$O(n)$	$O(h)$	$O(\lg n)$
Buscar valor arbitrario	$O(n)$	$O(h)$	$O(n)$
Altura garantizada	—	No	Sí, $O(\lg n)$

Comparando con lo que ya conoces

	Arreglo	**BST**	**Heap**
Ver mínimo	$O(n)$	$O(h)$	$O(1)$
Insertar	$O(1)/O(n)$	$O(h)$	$O(\lg n)$
Extraer mínimo	$O(n)$	$O(h)$	$O(\lg n)$
Buscar valor arbitrario	$O(n)$	$O(h)$	$O(n)$
Altura garantizada	—	No	Sí, $O(\lg n)$

Un heap **gana** en acceso al extremo garantizado, mucho más simple que un BST — pero **pierde** por completo la capacidad de búsqueda general.

Aplicaciones

¿Para qué sirve esto en la práctica?

¿Para qué sirve esto en la práctica?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Cola de prioridad**: el uso más directo — procesar siempre el elemento de mayor prioridad primero (sistemas operativos, simulaciones de eventos).

¿Para qué sirve esto en la práctica?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Cola de prioridad**: el uso más directo — procesar siempre el elemento de mayor prioridad primero (sistemas operativos, simulaciones de eventos).
- **Heapsort**: construir un heap y extraer el extremo repetidamente da un algoritmo de ordenamiento $O(n \lg n)$ *in-place* — lo veremos en la próxima clase.

¿Para qué sirve esto en la práctica?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- **Cola de prioridad**: el uso más directo — procesar siempre el elemento de mayor prioridad primero (sistemas operativos, simulaciones de eventos).
- **Heapsort**: construir un heap y extraer el extremo repetidamente da un algoritmo de ordenamiento $O(n \lg n)$ *in-place* — lo veremos en la próxima clase.
- Algoritmos de grafos como **Dijkstra** (más adelante en el curso) usan un heap para siempre procesar el nodo más cercano no visitado.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un heap es, siempre, un **árbol binario completo** — eso garantiza altura $O(\lg n)$ sin ningún balanceo extra.
- La **propiedad de heap** (min o max) es **local**: cada padre vs. sus hijos directos — no un orden total como en BST.
- Un árbol binario completo se representa **sin punteros**, en un arreglo — 0-indexado ($izq(i) = 2i + 1$, $der(i) = 2i + 2$) o 1-indexado ($izq(i) = 2i$, $der(i) = 2i + 1$) son igual de válidos; de aquí en adelante usamos 1-indexado.
- Insertar y Extraer-Extremo son $O(\lg n)$; Ver-Extremo es $O(1)$; Construir-Heap es $O(n)$ — todo esto se demuestra la próxima clase.

Logros de la sesión



Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos qué es un árbol binario completo y por qué garantiza altura $O(\lg n)$.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos qué es un árbol binario completo y por qué garantiza altura $O(\lg n)$.
- Sabemos enunciar la propiedad de Min-Heap / Max-Heap, y qué SÍ y qué NO garantiza.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos qué es un árbol binario completo y por qué garantiza altura $O(\lg n)$.
- Sabemos enunciar la propiedad de Min-Heap / Max-Heap, y qué SÍ y qué NO garantiza.
- Sabemos representar un árbol binario completo con un arreglo, usando las fórmulas de índices.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos qué es un árbol binario completo y por qué garantiza altura $O(\lg n)$.
- Sabemos enunciar la propiedad de Min-Heap / Max-Heap, y qué SÍ y qué NO garantiza.
- Sabemos representar un árbol binario completo con un arreglo, usando las fórmulas de índices.
- Conocemos el TDA Heap y la complejidad esperada de cada una de sus operaciones.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias